

Lecture 3: Merge Sort

Xinyu Sheng

xs354@yale.edu

BIS 527: Introduction to Health Data Science

Yale School of Public Health

Lecturer: Harsh Parikh • Fall 2026

TL;DR

Merge sort divides a list into two parts, sorts each part recursively, and merges the sorted parts. The procedure halts because every recursive call operates on a smaller list. Its correctness follows from the correctness of merging and strong induction. Its running time is $\Theta(n \log n)$. Dividing each list into three parts instead does not change this asymptotic cost. Merge sort improves on bubble sort because it uses logarithmically many levels of linear work rather than linearly many sweeps. The comparison-tree lower bound proves that no comparison sort can guarantee $O(n)$ worst-case time for arbitrary values.

AI use declaration. I used GPT-5.6 to research the topic and polish the writing in these notes.

1 Motivation

A clinic stores the heights of five patients as $\langle 171, 158, 180, 152, 163 \rangle$ centimetres. Ordering the patient records by height produces $\langle 152, 158, 163, 171, 180 \rangle$. Every recorded height must remain in the output. Repeated sweeps can sort this list, but their worst-case cost grows quadratically with the number of records (Astrachan, 2003; Cormen et al., 2022). Merge sort reduces that cost by breaking the task into smaller copies of itself and then combining their results (Cormen et al., 2022; Sedgewick and Wayne, 2011).

The combination needs an argument of its own. If $\langle 158, 171 \rangle$ and $\langle 152, 163, 180 \rangle$ have each been sorted, simply putting the first list before the second gives 171 before 152. A merge must interleave the two lists. The central questions are whether the complete procedure halts, whether it returns the required output, and how much time and space it costs.

2 Setup and Notation

Lists and slices. Let A be a finite list of n values drawn from a totally ordered set. Total order means that any two values can be compared and that the comparisons are consistent. Write $A[i : j]$ for positions $i, i + 1, \dots, j - 1$. The right endpoint is excluded, so $A[0 : n]$ is the whole list. Write $|A|$ for the length of A .

Definition 1 (The sorting problem). The input is a list A of n values from a totally ordered set. The output is a permutation B of those values such that $B[i] \leq B[i + 1]$ for every $0 \leq i < n - 1$. A permutation contains every input value with the same multiplicity, so it neither loses a value nor creates one.

Costs. A comparison tests one value against another. Extra space is storage used beyond the input list. For nonnegative functions f and g , the notation $f(n) \in O(g(n))$ means that constants $c, n_0 > 0$ exist such that $f(n) \leq cg(n)$ for every $n \geq n_0$. The notation $f(n) \in \Omega(g(n))$ gives the corresponding lower bound. Membership in both sets is written $f(n) \in \Theta(g(n))$. These notations describe growth as n increases rather than an exact number of seconds.

3 How Merge Sort Works

Take $A = \langle 8, 3, 2, 9, 7, 1, 5, 4 \rangle$. The first division produces $\langle 8, 3, 2, 9 \rangle$ and $\langle 7, 1, 5, 4 \rangle$. Further divisions produce pairs and then one-value lists. A one-value list is already sorted. The procedure then combines one-value lists into sorted pairs, sorted pairs into sorted lists of length four, and those lists into the final sorted output.

The combination of $L = \langle 2, 5, 8 \rangle$ and $R = \langle 1, 4, 7 \rangle$ illustrates the essential operation. Compare the first unused values, 2 and 1, and append 1 to the output. The next comparison is between 2 and 4, so 2 is appended. Continuing in this way produces $\langle 1, 2, 4, 5, 7, 8 \rangle$. Each comparison involves only the first unused value in each list. Since each input list is already sorted, those two values are the smallest values remaining in their respective lists.

This pattern is called *divide and conquer*. The division creates smaller instances of the original problem. The recursive calls solve those instances. For an input of length at least two, set $m := \lfloor n/2 \rfloor$. Recursively sort $A[0 : m]$ and $A[m : n]$. The two slices have lengths $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$. For an empty or one-value input, return a copy immediately.

The indices i and j identify the first unused values of L and R . The list B holds the output. Each pass through the first loop compares $L[i]$ with $R[j]$ and appends the smaller value. Once either list is exhausted, the last two loops append the unused values of the other list. Algorithm 1 writes slices for readability. They may be implemented either as copied lists or as index ranges in one array.

Algorithm 1 Merge sort with an explicit merge operation

```
1: procedure MERGESORT( $A$ )
2:    $n \leftarrow |A|$ 
3:   if  $n \leq 1$  then
4:     return a copy of  $A$ 
5:   end if
6:    $m \leftarrow \lfloor n/2 \rfloor$ 
7:    $L \leftarrow \text{MERGESORT}(A[0 : m])$ 
8:    $R \leftarrow \text{MERGESORT}(A[m : n])$ 
9:   return MERGE( $L, R$ )
10: end procedure

11: procedure MERGE( $L, R$ ) ▷  $L$  and  $R$  are sorted
12:    $i \leftarrow 0, j \leftarrow 0, B \leftarrow$  empty list
13:   while  $i < |L|$  and  $j < |R|$  do
14:     if  $L[i] \leq R[j]$  then
15:       append  $L[i]$  to  $B$ ;  $i \leftarrow i + 1$ 
16:     else
17:       append  $R[j]$  to  $B$ ;  $j \leftarrow j + 1$ 
18:     end if
19:   end while
20:   while  $i < |L|$  do
21:     append  $L[i]$  to  $B$ ;  $i \leftarrow i + 1$ 
22:   end while
23:   while  $j < |R|$  do
24:     append  $R[j]$  to  $B$ ;  $j \leftarrow j + 1$ 
25:   end while
26:   return  $B$ 
27: end procedure
```

4 Halting

A procedure must halt before its output can satisfy Definition 1. Both the merge loop and the recursive sorting calls require a termination argument. Strong induction matches the way their remaining input sizes decrease.

Theorem 1 (Termination). *MERGE and MERGESORT terminate for every pair of finite sorted lists and every finite input list, respectively.*

Proof. First use strong induction on the total number d of unused values in a call to MERGE. If $d = 0$, both lists are exhausted and the loops stop. Assume that every merge state with fewer than d unused values terminates. If one list is exhausted, a final loop appends the finite suffix of the other list and stops. If neither list is exhausted, the first loop appends one value and advances one

index. Exactly $d - 1$ unused values remain. The inductive hypothesis shows that the continuation terminates. Therefore MERGE terminates for every finite pair of inputs.

Now use strong induction on the input length n for MERGESORT. If $n \leq 1$, the procedure returns without a recursive call. Assume that MERGESORT terminates for every input of length less than n , where $n \geq 2$. The two recursive calls receive $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$ values. Both lengths are less than n , so both calls terminate by the inductive hypothesis. Their outputs are finite lists, so the subsequent merge terminates. Thus MERGESORT terminates on every input of length n . $\square$

5 Correctness

The correctness proof has two parts. The first part proves that the merge of two sorted lists is correct. The second part uses that result to prove that the entire recursive procedure is correct. Both parts use strong induction.

Lemma 2 (Merge lemma). *If L and R are sorted, MERGE returns a sorted permutation of their concatenation.*

Proof. Use strong induction on $n := |L| + |R|$, the total number of values to be merged. If $n = 0$, both lists are empty and the returned empty list is a sorted permutation of their concatenation. If $n = 1$, one list is empty and the other contains one value. The procedure appends that value and returns the required output.

For the inductive step, fix $n \geq 2$ and assume the claim for every total size less than n . If one list is empty, the procedure appends the other list. That list is already sorted and contains exactly the values in the two inputs. Now suppose both lists are nonempty. Their first unused values are $L[0]$ and $R[0]$. Since each list is sorted, these are the smallest values in their respective lists. The smaller of $L[0]$ and $R[0]$ is therefore the smallest value in both lists together.

The procedure appends that smallest value first. It then continues with the unused suffixes, which remain sorted and contain $n - 1$ values in total. The strong inductive hypothesis shows that those suffixes are merged into a sorted permutation of their values. The first appended value is no greater than any value in that remaining output. The complete output is therefore sorted. Exactly one original value was appended first, and the inductive hypothesis accounts for every remaining value with its original multiplicity. The complete output is also a permutation of the two inputs. $\square$

Theorem 3 (Correctness of merge sort). *For every $n \geq 0$ and every list A of length n , MERGESORT returns a sorted permutation of A .*

Proof. Use strong induction on n . Let $P(n)$ be the theorem's claim for every input of length n . If $n = 0$ or $n = 1$, the procedure returns a copy of A . Such a list is sorted and contains the original values.

For the inductive step, fix $n \geq 2$ and assume $P(k)$ for every $k < n$. The two recursive calls receive lists of lengths $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$. Both lengths are less than n . The strong inductive hypothesis shows that the two calls return sorted permutations of $A[0 : m]$ and $A[m : n]$. Lemma 2 shows that their merge is a sorted permutation of their concatenation. The two input slices partition A , so that concatenation contains every value of A with the same multiplicity. The merged result is therefore a sorted permutation of A . $\square$

6 Cost

Suppose MERGE receives lists of lengths a and b , where $a, b > 0$. Every comparison appends one value. The comparison loop stops when one list is exhausted. At that time at least one value remains for the final append, so at most $a + b - 1$ comparisons have occurred. Across all loops, exactly $a + b$ values are appended. One merge therefore takes $\Theta(a + b)$ time.

Let $T(n)$ be the worst-case running time for a length- n input. Sorting the two parts costs $T(\lfloor n/2 \rfloor)$ and $T(\lceil n/2 \rceil)$. Merging them costs $\Theta(n)$. Copying slices also fits within the linear term if the implementation copies them. Hence

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n), \quad T(0), T(1) = \Theta(1). \quad (1)$$

For $n = 2^h$, the recurrence becomes $T(n) = 2T(n/2) + \Theta(n)$. A recursion tree makes the sum visible. If $n = 8$, the top merge handles eight values. The next level has two merges of four values each, for a total of eight. The next level has four merges of two values each, again for a total of eight. In general, the subproblem sizes at any one level sum to n , so each level costs $\Theta(n)$.

Repeated halving gives the sizes $n, n/2, n/4, \dots, 1$. The number of halvings is $\log_2 n$, so the tree has $\Theta(\log n)$ merge levels. Multiplying the cost per level by the number of levels gives

$$T(n) \in \Theta(n) \Theta(\log n) = \Theta(n \log n). \quad (2)$$

When n is not a power of two, the two parts differ by at most one. Their maximum depth remains $\Theta(\log n)$, so the same bound holds. The standard merge sort performs the splitting and merging on every input. Its usual array implementation therefore has the same asymptotic best-case and worst-case running time.

An ordinary array implementation holds a temporary merge buffer of $\Theta(n)$ values. Its recursion stack holds $\Theta(\log n)$ frames. Peak extra space is therefore $\Theta(n)$ rather than constant. Different representations can change implementation details, but they do not change the time bound.

7 Worked Example

Take $A = \langle 171, 158, 180, 152, 163 \rangle$. The first division produces $\langle 171, 158 \rangle$ and $\langle 180, 152, 163 \rangle$. Recursive sorting produces $L = \langle 158, 171 \rangle$ and $R = \langle 152, 163, 180 \rangle$. The top-level merge proceeds as follows:

Values compared	Value appended	Output after append
158 and 152	152	$\langle 152 \rangle$
158 and 163	158	$\langle 152, 158 \rangle$
171 and 163	163	$\langle 152, 158, 163 \rangle$
171 and 180	171	$\langle 152, 158, 163, 171 \rangle$
Right remainder	180	$\langle 152, 158, 163, 171, 180 \rangle$

The top-level merge makes four comparisons. The final value needs no comparison because the left list is already exhausted. The output contains the five original heights in nondecreasing order, so it satisfies Definition 1.

8 Beyond the Lecture

Three parts. The division can produce three parts instead of two. Set $p := \lfloor n/3 \rfloor$ and $q := \lfloor 2n/3 \rfloor$. Recursively sort $A[0 : p]$, $A[p : q]$, and $A[q : n]$. A three-way merge compares the first unused values of the nonempty lists and appends the smallest. The reasoning in Lemma 2 extends because that value is the smallest among all values that remain. Strong induction also applies because every nonempty child has fewer than n values when $n \geq 2$.

At most two comparisons find the smallest of three candidate values. A complete three-way merge therefore takes $\Theta(n)$ time. For balanced thirds, the recurrence is

$$T_3(n) = T_3(n_1) + T_3(n_2) + T_3(n_3) + \Theta(n), \quad n_1 + n_2 + n_3 = n, \quad (3)$$

where each n_i is approximately $n/3$. If $n = 3^h$, there are $h = \log_3 n$ merge levels. Each level still processes all n values, so each level costs $\Theta(n)$. It follows that $T_3(n) \in \Theta(n \log_3 n) = \Theta(n \log n)$. The identity $\log_3 n = \log_2 n / \log_2 3$ shows that changing the logarithm's base changes only a constant factor. Three-way division changes the shape of the recursion tree but not the asymptotic cost.

Why merge sort costs less than bubble sort. One left-to-right sweep of bubble sort moves the largest unsorted value to its final position. The next call still has $n - 1$ values to sort. Its worst-case comparison count therefore satisfies

$$B(n) = (n - 1) + (n - 2) + \cdots + 1 = \frac{n(n - 1)}{2} \in \Theta(n^2). \quad (4)$$

Bubble sort has a linear number of sweeps, and a sweep can contain a linear number of comparisons. Merge sort instead halves every subproblem. Halving gives only a logarithmic number of levels. Although each merge level performs linear work, the product is $\Theta(n \log n)$ rather than $\Theta(n^2)$. The improvement comes from the balanced decomposition, not from making an individual merge cost less than one bubble sweep.

Can comparison sorting be linear? A *comparison sort* obtains order information only from comparisons between pairs of values. For three distinct values, there are $3! = 6$ possible input orders. One comparison has two possible outcomes, so two comparisons can distinguish at most $2^2 = 4$ cases. At least three comparisons are therefore required for some three-value input.

The same argument extends to n distinct values. A *comparison decision tree* represents one comparison at each internal node and one possible outcome on each of its two branches. Each leaf represents an order that the procedure has distinguished. There are $n!$ possible input orders, so a correct comparison sort needs at least $n!$ leaves. A binary tree of height h has at most 2^h leaves. Its height must therefore satisfy

$$2^h \geq n! \quad \text{and hence} \quad h \geq \log_2(n!). \quad (5)$$

At least $n/2$ factors in $n!$ are at least $n/2$. Consequently,

$$n! \geq \left(\frac{n}{2}\right)^{n/2} \implies \log_2(n!) \geq \frac{n}{2} \log_2\left(\frac{n}{2}\right) \in \Omega(n \log n). \quad (6)$$

The height of the tree is the number of comparisons on its longest path. Every comparison sort therefore requires $\Omega(n \log n)$ comparisons in the worst case. It cannot guarantee an $O(n)$ worst-case cost for arbitrary distinct values. Merge sort attains $\Theta(n \log n)$, so it is asymptotically optimal within the comparison model.

This lower bound does not apply to every possible sorting method. If the values are integers in a known range $\{0, 1, \dots, K-1\}$, *counting sort* uses the values as array indices. Its time is $\Theta(n + K)$. This becomes $\Theta(n)$ when $K \in O(n)$. Counting sort avoids the lower bound because it uses information other than pairwise comparisons.

9 Summary

Merge sort divides a list, sorts the smaller parts recursively, and merges their sorted outputs. Strong induction proves that the merge and the full recursive procedure both terminate. A second pair of strong-induction arguments proves that the output is a sorted permutation of the input. Linear work at each of logarithmically many levels gives $\Theta(n \log n)$ time. Three-way division has the same

asymptotic cost. Bubble sort costs $\Theta(n^2)$ in the worst case because it performs linearly many sweeps. The comparison-tree argument rules out worst-case $O(n)$ comparison sorting for arbitrary values.

Further Reading

[Cormen et al. \(2022\)](#) develop merge sort, divide-and-conquer recurrences, the comparison-sorting lower bound, and counting sort. [Sedgewick and Wayne \(2011\)](#) provide another treatment of mergesort and practical sorting implementations. [Knuth \(1998\)](#) gives a detailed analysis of merging and sorting. [Astrachan \(2003\)](#) examines bubble sort and its place in introductory computing. The examples and proof organization in this note were written for this chapter rather than reproduced from those sources.

References

- Astrachan, O. (2003). Bubble sort: An archaeological algorithmic analysis. *ACM SIGCSE Bulletin*, 35(1):1–5.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms*. MIT Press, Cambridge, MA, 4th edition.
- Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA, 2nd edition.
- Sedgewick, R. and Wayne, K. (2011). *Algorithms*. Addison-Wesley, Upper Saddle River, NJ, 4th edition.